CHAPTER 10

FUNDAMENTAL NETWORK ALGORITHMS

A discussion of some of the most important and fundamental algorithms for performing network calculations on a computer

ARMED WITH the tools and data structures of Chapter 9, we look in this chapter at the algorithms that are used to perform network calculations. We start with some simple algorithms for calculating quantities such as degrees, degree distributions, and clustering. In the later sections of the chapter we look at more sophisticated algorithms for shortest paths, betweenness, maximum flows, and other non-local quantities.

In the following chapter we extend our examination of network algorithms to algorithms based on matrix calculations and linear algebra, including algorithms for matrix-based centralities like eigenvector centrality and algorithms for graph partitioning and community discovery in networks.

10.1 ALGORITHMS FOR DEGREES AND DEGREE DISTRIBUTIONS

Many network quantities are easy to calculate and require only the simplest of algorithms, algorithms that are little more than translations into computer code of the definitions of the quantities in question. Nonetheless, it is worth looking at these algorithms at least briefly, for two reasons. First, there is in some cases more than one simple algorithm for calculating a quantity, and one algorithm may be much faster than another. It pays to evaluate one's algorithm at least momentarily before writing a computer program, to make sure one is going about the calculation in the most sensible manner. Second, it is worthwhile to calculate the computational complexity of even the simplest algorithm, so that one can make an estimate of how long a computation will take to finish—see Section 9.1. Even simple algorithms can take a long time to run.

One of the most fundamental and important of network quantities is the degree of a vertex. Normally degrees are very simple to calculate. In fact, if a network is stored in the form of an adjacency list[144] then, as described in Section 9.4, we normally maintain an array containing the degree of each vertex so that we know how many entries there are in the list of neighbors for each vertex. That means that finding the degree of any particular vertex is a simple matter of looking it up in this array, which takes O(1) time.

If the network is stored in an adjacency matrix, then the calculation takes longer. Calculating the degree of a vertex i in this case involves going through all elements of the ith row of the adjacency matrix and counting the number that are non-zero. Since there are n elements in each row of the matrix, where n is the number of vertices in the network, the calculation takes time $O(n)$, making the calculation far slower than for the adjacency list. If one needed to find the degrees of vertices frequently during the course of a larger calculation using an adjacency matrix, it might make good sense to calculate the degree of each vertex once and for all and store the results for later easy retrieval in a separate array.

In Section 8.3 we discussed degree distributions, which are of considerable interest in the study of networks for the effect they have on network structure and processes on networks (see Chapters 13 and 14). Calculating a degree distribution p_k is also very straightforward: once we have the degrees of all vertices, we make a histogram of them by creating an array to store the number of vertices of each degree up to the network maximum, setting all the array elements initially to zero, and then running through the vertices in turn, finding the degree k of each and incrementing by one the kth element of the array. This process trivially takes time $O(n)$ to complete. Once it is done the fraction p_k of vertices of degree k is given by the count in the kth array element divided by n.

The cumulative distribution function P_k of Section 8.4.1 requires a little more work. There are two common ways to calculate it. One is first to form a histogram of the degrees as described above and then to calculate the cumulative distribution directly from it using

$$P_k = \sum_{k'=k}^{\infty} p_{k'} = -p_{k-1} + \sum_{k'=k-1}^{\infty} p_{k'} = P_{k-1} - p_{k-1}, \tag{10.1}$$

Noting that $P_0 = \sum_{k'=0}^{\infty} p_{k'} = 1$, we can then start from P_0 and use Eq. (10.1) to calculate successive

P_k up to any desired value of k. This process trivially takes $O(n)$ time and, since the calculation of p_k also takes $O(n)$ time, the whole process is $O(n)$.

In fact, however, as described in Section 8.4.1, this is not usually how one calculates the cumulative distribution function. Although the method is fast, it's also moderately complicated and there is a simpler way of doing the calculation that involves taking the degrees of all the vertices, sorting them in descending order, and ranking them from 1 to n. A plot of the rank divided by n as a function of degree then gives the cumulative distribution. The most time-consuming part of this calculation is the sorting of the degrees. Sorting is a well-studied problem and the fastest general algorithms[145] run in time $O(n \log n)$. Thus the leading order scaling of this algorithm to calculate the cumulative distribution is $O(n \log n)$. This is slower than the first method described above, which was $O(n)$, but not much slower and the second method has the considerable advantage that almost all computers provide standard software for sorting numbers, which means that in most cases one doesn't have to write a program at all to calculate the cumulative distribution. All spreadsheet programs, for instance, include facilities for sorting numbers, so one can calculate cumulative distributions directly in a spreadsheet.

Another quantity of interest is the correlation coefficient r for vertex degrees, Eq. (8.26), which measures assortative mixing by degree. This too is straightforward to calculate—one uses Eq. (8.27) and the sums defined in Eqs. (8.28) and (8.29). Given the degrees of all vertices, the sum in Eq. (8.28) takes time $O(m)$ to evaluate, where m is the number of edges in the network, and the sums in Eq. (8.29) each take time $O(n)$, so the total time required to calculate r is $O(m + n)$. As mentioned in Section 9.1, we are often concerned with sparse networks in which the mean degree remains constant as the network gets larger, i.e., networks in which $m \propto n$. In such networks $O(m + n) \equiv O(n)$ and the time to calculate r taken just scales as the number of vertices. On the other hand, if the network is dense, meaning that $m \propto n^2$, then $O(m) \equiv O(n^2)$, which is considerably worse.

10.2 CLUSTERING COEFFICIENTS

The calculation of clustering coefficients is only slightly more complicated than the calculation of degrees. To see how it works, we start by calculating the local clustering coefficient C_i for a single vertex i on an undirected network, Eq. (7.42):

$$C_i = \frac{\text{(number of pairs of neighbors of } i \text{ that are connected)}}{\text{(number of pairs of neighbors of } i\text{)}}.$$

(10.2)

Calculating the numerator involves going through every pair of distinct neighbors of vertex i and counting how many are connected. We need only consider each pair once, which we can do conveniently by restricting ourselves to pairs (j, l) for which $j < l$. For each pair we determine whether an edge exists between them, which is done in various ways depending on the representation used for the network as described in Sections 9.3-9.5, and count up the number of such edges. Then we divide the result by the number of pairs, which is just $\frac{1}{2}k_i(k_i-1)$, where k_i is the degree of the vertex.

To calculate the overall clustering coefficient for the entire network, which is given by

$$C = \frac{\text{(number of triangles)} \times 3}{\text{(number of connected triples)}}.$$

(10.3)

(see Eq. (7.41)), we extend the same calculation to the whole network. That is we consider for every vertex each pair of neighbors (j, l) with $j < l$ and find whether they are connected by an edge.[147] We add up the total number of such edges over all vertices and then divide by the number of connected triples, which is $\sum_i \frac{1}{2}k_i(k_i-1)$.

This last algorithm is simple and straightforward, a direct implementation of the formula (10.3) defining the clustering coefficient, but some interesting issues nonetheless come up when we consider its running time. Even without performing a full calculation of the complexity of the algorithm we can see that something unusual is going to happen because a vertex i with degree k_i has $\frac{1}{2}k_i(k_i-1)$ pairs of neighbors. We have to check for the presence of an edge between each such pair on the entire network and hence the total number of checks we have to perform is

$$\sum_i \tfrac{1}{2}k_i(k_i-1) = \tfrac{1}{2}n\bigl(\langle k^2\rangle - \langle k\rangle\bigr),$$

(10.4)

where

$$\langle k \rangle = \frac{1}{n}\sum_i k_i, \qquad \langle k^2 \rangle = \frac{1}{n}\sum_i k_i^2,$$

(10.5)

are the mean and mean square degree for the network. (We previously denoted the mean degree by c, but we use the alternate notation $\langle k \rangle$ here for clarity, and to highlight the distinction between the mean and the mean square.)

See Section 8.3 for a discussion of degree distributions.

The interesting point here is that Eq. (10.4) depends in a non-trivial way on the degree distribution of our network. The running times of other algorithms we have seen so far have depended on the number of vertices n and the number of edges m, and hence, indirectly, on the mean degree $\langle k \rangle = 2m / n$. For the clustering coefficient, however, we see that the amount of work we have to do, and hence also the running time, depends not only on n and $\langle k \rangle$, but on the second moment $\langle k^2 \rangle$, which is an additional independent parameter. Even if we suppose that the degree distribution remains the same with increasing n so that the quantities $\langle k \rangle$ and $\langle k^2 \rangle$ can be considered constant, strange things can happen. Consider the case of a network whose degree distribution follows a power law $p_k \sim k^{-\alpha}$, as described in Section 8.4. For such networks, the first moment is well behaved but the second moment $\langle k^2 \rangle$ formally diverges if $\alpha < 3$ (see Section 8.4.2) which implies that it will take an infinite amount of time to evaluate the clustering coefficient!

To understand better what is going on let us perform a more careful calculation of the time complexity of our clustering coefficient algorithm. We start by considering again a single vertex i. And let us assume that we have our network stored in adjacency list form. In that case, we can, as we have seen, easily enumerate all of the neighbors of our vertex in time that goes like k_i. For each neighbor j we run through each other neighbor $l > j$ that could be paired with it, for a total of $\frac{1}{2}k_i(k_i-1)$ pairs and determine for each pair whether an edge exists between them. This latter operation takes a time proportional, to leading order, to either k_j or to k_l (see Table 9.2), depending on whether we find the edge by looking at the adjacency list for vertex j or for vertex l. Let us for the moment assume a simple algorithm that chooses at random between the two vertices, in which case the typical time taken will go as the average of the two degrees, i.e., it will be proportional to $k_j + k_l$.

Let Γ_i denote the set of neighbors of vertex i. Then the total time taken to check for edges between all pairs of neighboring vertices is proportional to

$$\begin{aligned}\sum_{j,l\in\Gamma_i : j<l} (k_j + k_l) &= \tfrac{1}{2}\sum_{j,l\in\Gamma_i : j\neq l} (k_j + k_l) = \sum_{j,l\in\Gamma_i : j\neq l} k_j \\ &= (k_i - 1)\sum_{j\in\Gamma_i} k_j.\end{aligned}$$

(10.6)

The total time needed to calculate the numerator of Eq. (10.3) is then proportional to the sum of this quantity over all vertices i:

$$\sum_i (k_i - 1) \sum_{j \in \Gamma_i} k_j = \sum_{ij} A_{ij}(k_i - 1)k_j = \sum_{ij} A_{ij} k_i k_j - \sum_j k_j^2$$

(10.7)

where A_{ij} is an element of the adjacency matrix and we have made use of the result $\sum_i A_{ij} = k_j$ (Eq. (6.19)).

Compare this equation with our earlier expression for the correlation coefficient r between degrees in a network, Eq. (7.82), which quantifies assortativity by degree in networks:

$$r = \frac{\sum_{ij}(A_{ij} - k_i k_j / 2m) k_i k_j}{\sum_{ij}(k_i \delta_{ij} - k_i k_j / 2m) k_i k_j}.$$

(10.8)

As we can see, the first term in Eq. (10.7) is the same as the first term in the numerator of the correlation coefficient. As a result, our estimate of the time to calculate the clustering coefficient depends on whether the degrees of vertices are correlated or not. This can lead to some interesting behaviors for specific networks, but for simplicity let us assume here that there is no correlation between degrees, that the network we are considering has no assortativity. In that case $r = 0$ in Eq. (10.8), which can only occur if the numerator is itself zero, or equivalently if

$$\sum_{ij} A_{ij} k_i k_j = \frac{1}{2m} \sum_{ij} k_i^2 k_j^2 = \frac{1}{2m} \Big[\sum_i k_i^2 \Big]^2.$$

(10.9)

Combining this result with Eq. (10.7), the running time for our calculation of the clustering coefficient on an uncorrelated network is proportional to

$$\frac{1}{2m} \Big[\sum_i k_i^2 \Big]^2 - \sum_j k_j^2 = n \langle k^2 \rangle \left[\frac{\langle k^2 \rangle}{\langle k \rangle} - 1 \right],$$

(10.10)

where we have made use of the fact that $2m = \sum_i k_j = n\langle k\rangle$ (see Eq. (6.20)).

This is a measure of the time taken to evaluate the numerator of Eq. (10.3). The denominator is simply equal to $\sum_i k_i(k_i - 1)$ and so just takes $O(n)$ time to evaluate, given that, for a network stored in adjacency list format, we already have the degrees of all vertices available. This will never be longer than the time represented in Eq. (10.10), so Eq. (10.10) gives the leading-order time complexity of the calculation of the clustering coefficient.

So we see that the calculation of the clustering coefficient indeed takes a time that depends not only on n and m but also on the second moment $\langle k^2\rangle$ of the degree distribution. In many cases this does not matter, since the second moment often tends to a modest constant value as the network becomes large. But for networks with highly skewed degree distributions $\langle k^2\rangle$ can become very large and in the case of a power-law degree distribution with exponent $\alpha < 3$ it formally diverges (see Section 8.4.2) and with it so does the expected running time of our algorithm.

More realistically, if the network is a simple graph with no multiedges, then the maximum allowed degree is $k = n$ and the degree distribution is cut off, which means that the second moment scales at worst as $n^{3-\alpha}$ (Eq. (8.22)) while the first moment remains constant. This in turn implies that the running time of our clustering coefficient algorithm on a scale-free network would go as $n \times n^{3-\alpha} \times n^{3-\alpha} = n^{7-2\alpha}$. For values of α in the typical range of $2 \le \alpha \le 3$ (Table 8.1), this gives running times that vary from a minimum of $O(n)$ for $\alpha = 3$ to a maximum of $O(n^3)$ for $\alpha = 2$. For the lower values of α this makes the calculation of the clustering coefficient quite arduous, taking a time that increases sharply as the network gets larger.

So can we improve on this algorithm? There are various possibilities. Most of the work of the algorithm is in the "find" operation to determine whether there is an edge between a given pair of vertices, and the algorithm will be considerably faster if we can perform this operation more efficiently. One simple (though memory-inefficient) method is to make use of the hybrid matrix/list data structure of Section 9.6, which can perform the find operation in constant time.[148] Even in this case, however, the number of find operations that must be performed is still equal to the number of connected triples in the network, which means the running time is given by Eq. (10.4), and hence still formally diverges on a network with a power-law degree distribution. On a simple graph for which the power law is cut off at $k = n$, it will go as $n^{4-\alpha}$, which ranges from $O(n)$ to $O(n^2)$ for values of α in the interesting range $2 \le \alpha \le 3$. This is better than our earlier algorithm, but still relatively poor for the lower values of α.

These difficulties are specific to the case of scale-free networks. In other cases there is usually no problem calculating the clustering coefficient quickly. Some alternative algorithms have been proposed for calculating approximate values of the clustering coefficient rapidly, such as the algorithm of Schank and Wagner [292], and these may be worth considering if you need to perform calculations on very large networks.

10.3 SHORTEST PATHS AND BREADTH-FIRST SEARCH

We now move on to some more complex algorithms, algorithms for calculating mostly non-local quantities on the networks, such as shortest paths between vertices. The study of each of these algorithms has three parts. Two are, as before, the description of the algorithm and the analysis of its running time. But now we also include a proof that the algorithm described performs the calculation it claims to. For the previous algorithms in this chapter such proofs were unnecessary; the algorithms were direct implementations of the equations defining the quantities calculated. As we move onto more complex algorithms, however, it will become much less obvious why those algorithms give the results they do, and to gain a full understanding we will need to examine their working in some detail.

The first algorithm we look at is the standard algorithm for finding shortest distances in a network, which is called *breadth-first search*.[149] A single run of the breadth-first search algorithm finds the shortest (geodesic) distance from a single source vertex s to *every other* vertex in the same component of the network as s. In some cases we want to know only the shortest distance between a single pair of vertices s, t, but there is no procedure known for calculating such a distance that is faster in the worst case than calculating the distances from s to every other vertex using breadth-first search and then throwing away all of the results except for the one we want.[150]

With only minor modifications, as we will describe, breadth-first search can also find the geodesic path one must take to realize each shortest distance and if there is more than one geodesic path, it can find all such paths. It works also on both directed and undirected networks, although our description will focus on the undirected case.

10.3.1 DESCRIPTION OF THE ALGORITHM

Breadth-first search finds the shortest distance from a given starting vertex *s* to every other vertex in the same component as *s*. The basic principle behind the algorithm is illustrated in Fig. 10.1. Initially we know only that *s* has distance 0 from itself and the distances to all other vertices are unknown. Now we find all the neighbors of *s*, which by definition have distance 1 from *s*. Then we find all the neighbors of *those* vertices. Excluding those we have already visited, these vertices must have distance 2. And *their* neighbors, excluding those we have already visited have distance 3, and so on. On every iteration, we grow the set of vertices visited by one step.

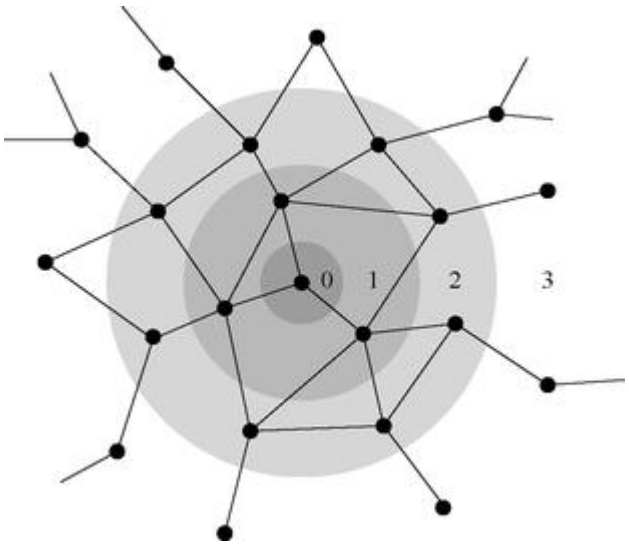


Figure 10.1: Breadth-first search. A breadth-first search starts at a given vertex, which by definition has distance 0 from itself, and grows outward in layers or waves. The vertices in the first wave, which are the immediate neighbors of the starting vertex, have distance 1. The neighbors of those neighbors have distance 2, and so forth.



A network path from *s* to *t* of length *d* (where *d* = 3 in this case) necessarily includes a path of length *d*–1 (i.e., 2) from *s* to an immediate neighbor of *t*.

This is the basic idea of breadth-first search. Now let us go over it more carefully to see how it works in practice and show that it really does find correct geodesic distances. We begin by noting the following fact:

> Every vertex whose shortest distance from s is d has a network neighbor whose shortest distance from s is $d - 1$.

This follows since if the shortest path from s to a vertex t is of length d then the penultimate vertex along that path, which is a neighbor of t, can, by definition, be reached in d - 1 steps and hence cannot have shortest distance greater than d -1. It also cannot have shortest distance less than d -1 because it if did there would be a path to t of length less than d.

Now suppose that we already know the distance to every vertex on the network that is d steps or less from our source vertex s. For example, we might know all the distances to vertices at distance 2 or less from the central vertex in Fig. 10.1. For every neighbor of one of the vertices at distance d there exists a path of length $d + 1$ to that neighbor: we can get to the vertex at distance d along a path of length d and then we take one more step to its neighbor. Thus every such neighbor is *at most* $d + 1$ steps from s, but it could be less than $d + 1$ from s if there is another shorter path through the network. However, we already know whether there is a shorter path to any particular vertex, since by hypothesis we know the distance to every vertex d steps or less from s.

Consider the set of all vertices that are neighbors of vertices at distance d but that are not already known to have distance d or less from s. We can say immediately that (1) all neighbors in this set have distance $d + 1$ from s, and (2) that there are no other vertices at distance $d + 1$. The latter follows from the property cited above: all vertices at distance $d + 1$ must be neighbors of vertices at distance d. Thus we have found the set of vertices at distance $d + 1$, and hence we now know the distances to all vertices that are $d + 1$ or less from s.

Now we just repeat the process. On each round of the algorithm we find all the vertices one step further out from s than on the last round. The algorithm continues until we reach a point at all the neighbors of vertices at distance d are found already to have known distances of d or less. This implies that there are no vertices of distance $d + 1$ and hence, by the property above, no vertices of any greater distance either, and so we must have found every vertex in the component containing s.

As a corollary of the process of finding distance, breadth-first search thus also finds the component to which vertex s belongs, and indeed breadth-first search is the algorithm of choice for finding components in networks.

10.3.2 A NAIVE IMPLEMENTATION

Let us now consider how we would implement breadth-first search on our computer. The simplest approach (but not, as we will see, the best) would go something like this. We create an array of n elements to store the distance of each vertex from the source vertex s, and initially set the distance of vertex s from itself to be zero while all other vertices have unknown distance from s. Unknown distances could be indicated, for instance, by setting the corresponding element of the array to −1, or some similar value that could never occur in reality.

We also create a distance variable d to keep track of where we are in the breadth-first search process and set its value initially to zero. Then we do the following:

1. Find all vertices that are distance d from s, by going through the distance array, element by element.
2. Find all the neighbors of those vertices and check each one to see if its distance from s is unknown (denoted, for example, by an entry −1 in the distance array).
3. If the number of neighbors with unknown distances is zero, the algorithm is over. Otherwise, if the number of neighbors with unknown distances is non-zero, set the distance of each of those neighbors to $d + 1$.
4. Increase the value of d by 1.
5. Repeat from step 1.

When the algorithm is finished we are left with an array that contains the distances to every vertex in the component of the network that contains s (and every vertex in every other component has unknown distance).

How long does this algorithm take? First of all we have to set up the distance array, which has one element for each vertex. We spend a constant amount of time setting up each element, so overall we spend $O(n)$ time setting up the distance array.

For the algorithm proper, on each iteration we go through all n vertices looking for those with distance d. Most will not have distance d in which case we pass over them, spending only O(1) time on each. Thus there is a basic cost of $O(n)$ time for each iteration. The total number of iterations we will for the moment call r, and overall we thus spend $O(rn)$ time on this part of the algorithm, in the worst case.

However, when we *do* come across a vertex with distance d, we must pause at that vertex and spend an additional amount of time checking each of its neighbors to see if their distances are unknown and assigning them distance $d + 1$ if they are. If we assume that the network is stored in adjacency list format (see Section 9.4) then we can go through the neighbors of a vertex in $O(m / n)$ on average, and during the whole course of the algorithm we pause like this at each vertex exactly once so that the total extra time we spend on checking neighbors of vertices is $n \times O(m / n) = O(m)$.

Thus the total running time of the algorithm, including set-up, is $O(n + rn + m)$.

And what is the value of the parameter r? The value of r is the maximum distance from our source vertex s to any other vertex. In the worst case, this distance is equal to the diameter of the network (Section 6.10.1) and the worst-case diameter is simply n, which is realized when the network is just a chain of n vertices strung one after another in a line. Thus in the worst case our algorithm will have running time $O(m + n^2)$ (where we have dropped the first n because we are keeping only the leading-order terms).

This is very pessimistic, however. As discussed in Sections 8.2 and 12.7 the diameter of most networks increases only as $\log n$, in which case our algorithm would run in time $O(m + n \log n)$ to leading order. This may be a moot point, however, since we can do significantly better than this if we use a little cunning in the implementation of our algorithm.

10.3.3 A BETTER IMPLEMENTATION

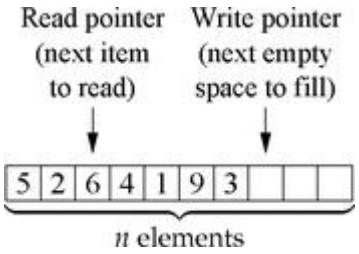
The time-consuming part of the implementation described in the previous section is step 1, in which we go through the list of distances to find vertices that are distance d from the starting vertex s. Since this operation involves checking the distances of all n vertices, only a small fraction of which will be at distance d, it wastes a lot of time. Observe, however, that in each wave of the breadth-first search process we find and label all vertices with a given distance $d + 1$. If we could store a list of these vertices, then on the next wave we wouldn't have to search through the whole network for vertices at distance $d + 1$; we could just use our list.

The most common implementation of this idea makes use of a *first-in/firstout buffer* or *queue*, which is nothing more than an array of (in this case) n elements that store a list of labels of vertices. On each sweep of the algorithm, we read the vertices with distance d from the list, we use these to find the vertices with distance $d + 1$, add those vertices with distance $d + 1$ to the list, and repeat.

To do this in practice, we fill up the queue array starting from the beginning. We keep a pointer, called the write pointer, which is a simple integer variable whose value indicates the next empty location at the end of the queue that has not been used yet. When we want to add an item to the queue, we store it in the element of the array pointed to by the write pointer and then increase the pointer by one to point to the next empty location.

At the same time we also keep another pointer, the read pointer, which points to the next item in the list that is to be read by our algorithm. Each item is read only once and once it is read the read pointer is increased by one to point to the next unread item.

Here is a sketch of the organization of the queue:



Our breadth-first search algorithm now uses two arrays of n elements, one for the queue and one for the distances from s to each other vertex. The algorithm is as follows.

1. Place the label of the source vertex s in the first element of the queue, set the read pointer to point to it, and set the write pointer to point to the second element, which is the first empty one. In the distance array, record the distance of vertex s from itself as being zero and the distances to all other vertices as "unknown" (for instance, by setting the corresponding elements of the distance array to -1, or some similar impossible value).
2. If the read and write pointers are pointing to the same element of the queue array then the algorithm is finished. Otherwise, read a vertex label from the element pointed to by the read pointer and increase that pointer by one.
3. Find the distance d for that vertex by looking in the distance array.
4. Go through each neighboring vertex in turn and look up its distance in the distance array as well. If it has a known distance, leave it alone. If it has an unknown distance, assign it distance $d + 1$, store its label in the queue array in the element pointed to by the write

pointer, and increase the write pointer by one.
5. Repeat from step 2.

Note the test applied in step 2: if the read pointer points to the same element as the write pointer, then there is no vertex to be read from the queue (since the write pointer always points to an empty element). Thus this test tells us when there are no further vertices waiting to have their neighbors investigated.

Note also that this algorithm reads all the vertices with distance d from the queue array one after another and uses them to find all the vertices with distance $d + 1$. Thus all vertices with the same distance appear one after another in the queue array, with the vertices of distance $d + 1$ immediately after those of distance d. Furthermore, each vertex appears in the queue array at most once. A vertex may of course be a neighbor of more than one other, but a vertex is assigned a distance and put in the queue only on the first occasion on which it is encountered. If it is encountered again, its distance is known rather than unknown, and hence it is not again added to the queue. Of course, a vertex may not appear in the queue array at all if it is never reached by the breadth-first search process, i.e., if it belongs to a different component from s.

Thus the queue does exactly what we wanted it to: it stores all vertices with a specified distance for us so that we have the list handy on the next sweep of the algorithm. This spares us from having to search through the network for them and so saves us a lot of time. In all other respects the algorithm works exactly as in the simple implementation of Section 10.3.2 and gives the same answers.

How long does this implementation of the algorithm take to run? Again there is an initial time of $O(n)$ to set up the distance array (see Section 10.3.2). Then, for each element in the queue, which means for each of the vertices in the same component as s, we do the following operations: we run through its neighbors, of which there are $O(m / n)$ on average, and either calculate their distance and add them to the queue, or do nothing if their distance is already known. Either way the operations take $O(1)$ time. Thus for each vertex in the component, of which there are in the worst case n, we spend time $O(m / n)$ and hence we require overall at most a time $n \times O(m / n) = O(m)$ to complete the algorithm for all n vertices.

Thus, including the time to set up the distance array, the whole algorithm takes time $O(m + n)$, which is better than the $O(m + n \log n)$ of the naive implementation (Section 10.3.2). For the common case of a sparse network with $m \propto n$, $O(m + n)$ is equivalent to $O(n)$ and our algorithm runs in time proportional to the number of vertices.[151] This seems just about optimal, since the algorithm is calculating the distance of all n vertices from the source vertex s. Thus it is assigning n numbers to the n elements of the distance array, which in the best possible case must take $O(n)$ time.

On a sparse network, therefore, the breadth-first search algorithm does as well as we can hope for in finding the distances from a single vertex to all others, and indeed it is the fastest known algorithm for performing this operation.

10.3.4 VARIANTS OF BREADTH-FIRST SEARCH

There are a number of minor variants of breadth-first search that merit a mention. First, one might wish to calculate the shortest distance between only a single pair of vertices s and t, rather than between s and all others. As mentioned in Section 10.3 there is no known way to do this faster than using breadth-first search. We can, however, improve the running time slightly by the obvious tactic of stopping the algorithm as soon as the distance to the target vertex t has been found. There is no point in continuing to calculate distances to the remaining vertices once we have the answer we want. In the worst case, the calculation still takes $O(m + n)$ time since, after all, our particular target vertex t might turn out to be the last one the algorithm finds. If we are lucky, however, and encounter the target early then the running time might be considerably shorter.

Conversely, we sometimes want to calculate the shortest distance between every pair of vertices in an entire network, which we can do by performing a breadth-first search starting at each vertex in the network in turn. The total running time for this "all-pairs shortest path" calculation is $n \times O(m + n) = O(n(m + n))$, or $O(n^2)$ on a sparse graph. As with the standard breadth-first search, this is optimal in the sense that we are calculating $O(n^2)$ quantities in $O(n^2)$ time, which is the best we can hope for.

As mentioned in the previous section, breadth-first search can also be used to identify the members of the component to which a vertex s belongs. At the end of the algorithm the distance array contains the distance from s to every vertex in its component, while distances to all other vertices are recorded as unknown. Thus we can find the size of the component just by counting the number of vertices with known distances. It takes time $O(n)$ to perform the count, so the operation of finding the component still takes $O(m + n)$ time in total.

The closeness centrality of Section 7.6 can also be calculated simply using breadth-first search. Recall that closeness is defined as the inverse of the mean distance from a vertex to all others in the same component. Since our breadth-first search calculates distances to all others in a component we need then only go through the distance array, calculate the sum of all known distances, divide by the size of the component, and take the inverse. Again the running time is $O(n + m)$. The variant closeness defined in terms of the harmonic mean in Eq. (7.30) can also be calculated, in the same running time, by a similar method.

10.3.5 FINDING SHORTEST PATHS

The breadth-first search algorithm as we have described it so far finds the shortest distance from a vertex *s* to all others in the same component of the network. It does not tell us the particular path or paths by which that shortest distance is achieved. With only a relatively small modification of the algorithm, however, we can calculate the paths as well. The trick is to construct another network on top of our original network, this one directed, that represents the shortest paths. This other network is often called the *shortest path tree*, although in the most general case it is a directed acyclic graph, not a tree.

The idea is as follows. At the start of our algorithm we create an extra network, which will become our shortest path tree, with the same number *n* of vertices as our original network and the same vertex labels, but with no edges at all. Then we start our breadth-first search algorithm from the specified source vertex *s* as before. The algorithm repeatedly pulls a vertex out of the queue and examines its neighbors, as described in Section 10.3.3, but now every time the neighbor *j* of some vertex *i* turns out to be a previously unseen vertex, one whose distance is recorded as "unknown," we not only assign *j* a distance and store it in the queue, we also add a directed edge to our shortest path tree *from* vertex *j to* vertex *i*. This directed edge tells us that we found *j* by following a path from its neighbor *i*. However, vertex *i* will also have a directed edge leading from it to one of its neighbors, telling us that we found *i* by following *that* path, and so forth. Thus, by following a succession of these directed edges we eventually get all the way back to *s*, and so we can reconstruct the entire shortest path between *j* and *s*.

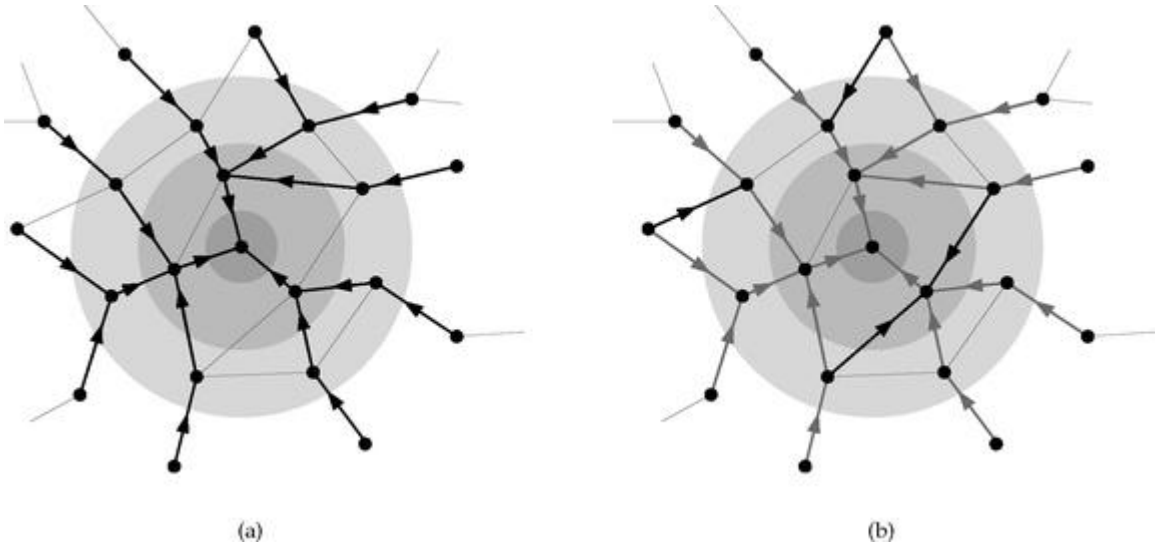


Figure 10.2: Shortest path trees. (a) A simple shortest path tree for the network of Fig. 10.1. Each vertex has a directed edge pointing to the vertex by which it was reached during the breadth-first search process. By following directed edges from any vertex we can find a shortest path to the starting vertex in the center. (b) The full shortest path tree (which is actually not a tree at all but a directed acyclic graph) contains extra directed edges that allow us to reconstruct all possible shortest paths.

So, when our breadth-first search is finished, the shortest path tree contains the information we

need to find the actual shortest path from every vertex in the component containing s to s itself. An example of a shortest path tree is shown in Fig. 10.2a for the same network as in Fig. 10.1.

This algorithm works well and the extra step of adding an edge to the shortest path tree can be accompanied quickly—in O(1) time if we store the network in adjacency list format (see Table 9.2). Thus the overall running time of the algorithm is still $O(m + n)$ to find all distances from s and the corresponding shortest paths.

The algorithm does have one shortcoming, however, which is that it only finds *one* shortest path to each vertex. As pointed out in Section 6.10.1, a pair of vertices may have more than one shortest path between them (see Fig. 6.10). Another slight modification of the algorithm allows us to deal with this case.

Multiple shortest paths exist between any vertex and the source vertex s if the path to s splits in two or more directions at some point along its length. This occurs if there is a vertex j somewhere along that path, say at distance $d + 1$ from s, that has more than one neighbor at distance d—see Fig. 10.2b. We can record this circumstance in our shortest-path tree by adding more than one directed edge from j to each of the relevant neighbors. These directed edges tell us that we can find a shortest path to vertex s by taking a step to any of those neighboring vertices.

To do this we modify our algorithm as follows. We perform the breadth-first search starting from s as before, and add directed edges from newly found vertices to their neighbor as before. But we also add an extra step. If, in the process of examining the neighbors of a vertex i that has distance d from the source vertex, we discover a neighbor j that already has an assigned distance, and that distance is $d + 1$, then we know that a path of length $d + 1$ has already been found to j, but we also know that *another* path of length $d + 1$ must exist via the current vertex i. So we add an extra directed edge to the shortest path tree from j to i. This makes the shortest path tree no longer a tree but, as we have said, it's usually called a tree anyway. In any case, the algorithm gives exactly what we want. When it is finished running the shortest path "tree" allows us to reconstruct all shortest paths from every vertex in the component to the source vertex s. See Fig. 10.2b.

10.3.6 BETWEENNESS CENTRALITY

In Section 7.7 we described betweenness centrality, a widely used centrality index that measures the extent to which a vertex in a network lies on the paths between other vertices. The betweenness centrality of vertex v is the number of geodesic paths between pairs of vertices s, t that pass through v. (Sometimes it is normalized to be the fraction of such paths, rather than the total number. The difference is only a multiplicative constant—see Section 7.7.) Given that we have a method for finding the shortest path (or paths) between any two vertices (Section 10.3.5), we can with only a little more work now create an algorithm for calculating betweenness.

The simplest way to calculate betweenness would be to implement the definition of the measure directly: use breadth-first search to find the shortest path between s and t, as described in Section 10.3.5 (assuming such a path exists), and then work our way along that path checking the vertices it passes though to see if the vertex v we are interested in lies among them. Repeating this process for every distinct pair s, t, we can then count the total number of paths that pass through v. (Things are slightly more complicated for the case in which a pair of vertices are connected by more than one shortest path, but let us ignore this complication for the moment—we will come to it soon.)

This algorithm is certainly a correct algorithm and it would work, but it is also inefficient. As we have seen, breadth-first search takes time $O(m + n)$ to find a shortest path between two vertices, and there are $\frac{1}{2}n(n-1)$ distinct pairs of vertices s, t. Thus the work of calculating betweenness for a single vertex would take $O(n^2 (m + n))$ time, or $O(n^3)$ in the common case of a sparse graph for which $m \propto n$. (The operation of checking the vertices along each shortest path will take time of the order of the length of the path, which is typically $O(\log n)$ (Section 8.2), making it negligible compared with the time taken to find the path.) This is prohibitively slow: while one might be able to calculate the betweenness of a vertex on a given network in, say, an hour's work, the same calculation on a graph ten times larger would take $10^3 = 1000$ hours, or more than a month of computer time.

But we can do a lot better if we make use of some of our results about breadth-first search from previous sections. First, the standard breadth-first search can find paths between a source s and all other vertices (in the same component) in time $O(m + n)$, which means, as noted in Section 10.3.4, we can find paths between all pairs in the network in time $O(n(m + n))$, or $O(n^2)$ on a sparse network.

An improved algorithm for calculating the betweenness of a vertex v might work as follows. For each s we use breadth-first search to find shortest paths between s and all other vertices, constructing a shortest path tree as described in Section 10.3.5. Then we use that tree to trace the paths from each vertex back to s, counting in the process the number of paths that go through v. We repeat this calculation for all s and so end up with a count of the total number of shortest paths that pass through v.

Indeed, we can trivially extend this algorithm to calculate betweenness for *all* vertices at the same time—we simply maintain a count of the number of paths that go through every vertex, for example in an array.[152]

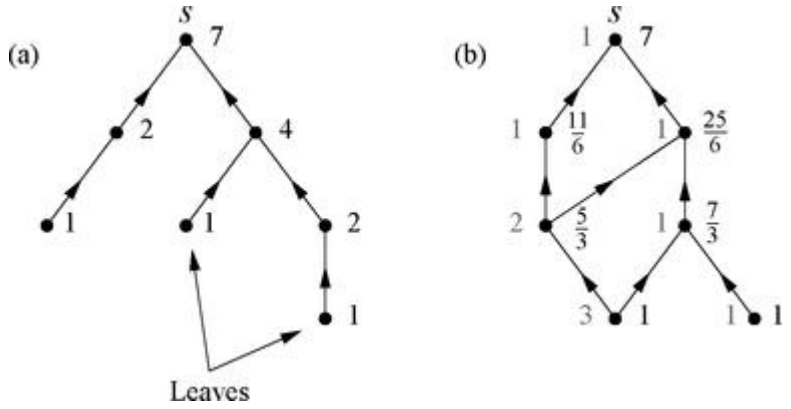


Figure 10.3: Calculation of betweenness centrality. (a) When there is only a single shortest path from a source vertex *s* (top) to all other reachable vertices, those paths necessarily form a tree, which makes the calculation of the contribution to betweenness from this set of paths particularly simple, as described in the text. (b) For cases in which there is more than one shortest path to some vertices, the calculation is more complex. First we must calculate the number of paths from the source *s* to each other vertex (numbers to the left of vertices), and then use these to weight the path counts appropriately and derive the betweenness scores (numbers to the right of vertices).

For any given *s*, this algorithm will take time $O(m + n)$ to find the shortest paths. Paths have length that by definition is less than or equal to the diameter of the network, which is typically of order $\log n$, and hence traversing the n paths from each vertex to *s* will take time $O(n \log n)$, for a running time of $O(m + n \log n)$ for each value of *s*. Repeating for all *s*, the whole algorithm will then take total time $O(n(m + n \log n))$ or $O(n^2 \log n)$ on a sparse network.

This is much better than our earlier $O(n^3)$ algorithm, but we can do better still. It is in fact possible to cut the running time down to just $O(n(m + n))$ by exploiting the fact that many of the shortest paths in the shortest path tree share many of the same edges. To understand this development, consider Fig. 10.3a, which shows a shortest path tree from a vertex *s* to all other vertices on a graph. In this case the shortest path tree really is a tree, meaning there is only one shortest path from *s* to any other vertex. This case is a good first example to study because of its simplicity, but we will consider the more general case in just a moment.

We use the tree to calculate a score for each vertex representing the number of shortest paths passing through that vertex. We find first the "leaves" of the tree, i.e., those vertices such that no shortest paths from other vertices to *s* pass through them. (In Fig. 10.3a the leaves are drawn at the bottom of the tree.) We assign a score of 1 to each of these leaves—the only path to *s* that passes through these vertices is the one that starts there.[153] Then, starting at the bottom of the tree we work upward, assigning to each vertex a score that is 1 plus the sum of the scores on the neighboring vertices immediately below it. That is, the number of paths through a vertex *v* is 1 for the path that starts at *v* plus the count of all paths that start below *v* in the tree and hence have to pass through it.

When we have worked all the way up the tree in this manner and reached vertex *s*, the scores at each vertex are equal to the betweenness counts for paths that end at vertex *s*. Repeating the process for all *s* and summing the scores, we arrive at the full betweenness scores for all paths.

In practice, the process of working up the tree can be accomplished by running through the vertices in order of decreasing distance from *s*. Conveniently, we already have a list of vertices in order of their distances, namely the entries in the queue array created by the breadth-first search process. Thus the betweenness algorithm in practice involves running backwards through the list of vertices in this array and calculating the number of paths through each vertex as above until the beginning of the array is reached.

In the worst case, this process involves going through all n vertices and checking every neighbor of every vertex, of which there are a total of $2m$, so that the overall running time is $O(m + n)$. The breadth-first search itself also takes time $O(m + n)$ (as usual) and hence the total time to count paths for each source vertex s is $O(m + n)$, which means the complete betweenness calculation takes time $O(n(m + n))$, as promised.

In general, however, we cannot assume that the shortest paths to a given vertex form a tree. As we saw in Section 10.3.5, often they do not. Consider, for instance, the "tree" shown in Fig. 10.3b. Following the definition of betweenness in Section 7.7, multiple shortest paths between the same pair of vertices are given equal weights summing to 1, so that for a vertex pair connected by three shortest paths, for example, we give each path weight $\frac{1}{3}$. Note that some of the paths may share vertices for part of their length, resulting in vertices with greater weight.

To calculate correctly the weights of the paths flowing through each vertex in a network, we need first to calculate the total number of shortest paths from each vertex to s. This is actually quite straightforward to do: the shortest paths from s to a vertex i must pass through one or more neighbors of i and the total number of shortest paths to i is simply the sum of the numbers of shortest paths to each of those neighbors. We can calculate these sums as part of a modified breadth-first search process as follows.

Consider Fig. 10.3b and suppose we are starting at vertex s. We carry out the following steps:

1. Assign vertex s distance zero, to indicate that it is zero steps from itself, and set $d = 0$. Also assign s a weight $w_s = 1$ (whose purpose will become clear shortly).
2. For each vertex i whose assigned distance is d, follow each attached edge to the vertex j at its other end and then do one of the following three things:
    a. If j has not yet been assigned a distance, assign it distance $d + 1$ and weight $w_j = w_i$.
    b. If j has already been assigned a distance and that distance is equal to $d + 1$, then the vertex's weight is increased by w_i, that is $w_j \leftarrow w_j + w_i$.
    c. If j has already been assigned a distance less than $d + 1$, do nothing.
3. Increase d by 1.
4. Repeat from step 2 until there are no vertices that have distance d.

The resulting weights for the example of Fig. 10.3b are shown to the left of each vertex in the figure. Each weight is the sum of the ones above it in the "tree." (It may be helpful to work through this example yourself by hand to see how the algorithm arrives at these values for the weights.) Physically, the weight on a vertex i represents the number of distinct geodesic paths between the source vertex s and i.

Now if two vertices i and j are connected by a directed edge in the shortest path "tree" pointing from j to i, then the fraction of the paths to s that pass through (or starting at) j and that also pass through i is given by w_i / w_j.

Thus, and finally, to calculate the contribution to the betweenness from shortest paths starting at all vertices and ending at s, we need only carry out the following steps:

1. Find every "leaf" vertex t, i.e., a vertex such that no paths from s to other vertices go though t, and assign it a score of $x_t = 1$.
2. Now, starting at the bottom of the tree, work up towards s and assign to each vertex i a score $x_i = 1 + \sum_j x_j w_i / w_j$, where the sum is over the neighbors j immediately below vertex i.
3. Repeat from step 2 until vertex s is reached.

The resulting scores are shown to the right of each vertex in Fig. 10.3b. Now repeating this process for all n source vertices s and summing the resulting scores on the vertices gives us the total betweenness scores for all vertices in time $O(n(m + n))$.[154]

This algorithm again takes time $O(n(m + n))$ in general or $O(n^2)$ on a sparse network, which is the best known running time for any betweenness algorithm at the time of writing, and moreover seems unlikely to be beaten by any future algorithm given that the calculation of the betweenness necessarily requires us to find shortest paths between all pairs of vertices, which operation also has time complexity $O(n(m + n))$. Indeed, even if we want to calculate the betweenness of only a single vertex it seems unlikely we can do better given that such a calculation still requires us to find all shortest paths.

10.4 SHORTEST PATHS IN NETWORKS WITH VARYING EDGE LENGTHS

In Section 6.3 we discussed weighted networks, networks in which the edges have values or strengths representing, for instance, the traffic capacities of connections on the Internet or the frequencies of contacts between acquaintances in a social network. In some cases the values on edges can be interpreted as lengths for the edges. The lengths could be real lengths, such as distances along roads in a road network, or they could represent quantities that act like lengths, such as transmission delays for packets traveling along Internet connections. In other cases they might just be approximately length-like measures: one might say, for instance, that a pair of acquaintances in a social network are twice as far apart as another pair if they see one another half as often.

Sometimes with networks such as these we would like to calculate the shortest path between two vertices taking the lengths of the edges into account. For instance, we might want to calculate the shortest driving route from A to B via a road network or we might want to calculate the route across the Internet that gets a data packet to its destination in the shortest time. (In fact, this is exactly what many Internet routers do when routing data packets.)

But now we notice a crucial—and annoying—fact. The shortest path across a network when we take edge lengths into account may not the be same as the shortest path in terms of number of edges. Consider Fig. 10.4. The shortest path between s and t in this small network traverses four edges, but is still shorter, in terms of total edge length, than the competing path with just two edges. Thus we cannot find the shortest path in such a network using standard breadth-first search, which finds paths with the minimum number of edges. For problems like this we need a different algorithm. We need *Dijkstra's algorithm*.

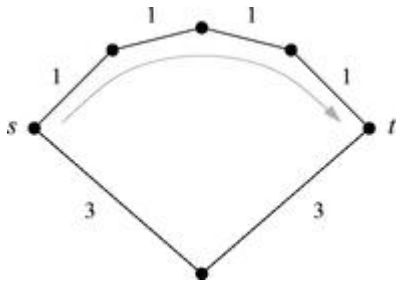


Figure 10.4: The shortest path in a network with varying edge lengths. The numbers on the edges in this network represent their lengths. The shortest path between s and t, taking the lengths into account, is the upper path marked with the arrow (which has total length 4), even though it traverses more edges than the alternative, lower path (which has length 6).

Dijkstra's algorithm, like breadth-first search, finds the shortest distance from a given source vertex s to every other vertex in the same component of a network, but does so taking the lengths of edges into account.[155] It works by keeping a record of the shortest distance it has found so far to each vertex and updating that record whenever a shorter one is found. It can be shown that, at the end of the algorithm, the shortest distance found to each vertex is in fact the shortest distance possible by any route. In detail the algorithm is as follows.

We start by creating an array of n elements to hold our current estimates of the distances from s

to every vertex. At all times during the running of the algorithm these estimates are upper bounds on the true shortest distances. Initially we set our estimate of the distance from s to itself to be zero, which is trivially correct, and from s to every other vertex to be ∞, which is clearly a safe upper bound.

We also create another array of n elements in which we record whether we are certain that the distance we have to a given vertex is the smallest possible distance. For instance, we might use an integer array with 1s to indicate the distances we are sure about and 0s for the distances that are just our best current estimate. Initially, we put a 0 in every element of this array. (You might argue that we know for certain that the distance from s to itself is zero and hence that we should put a 1 in the element corresponding to vertex s. Let us, however, pretend that we don't know this to begin with, as it makes the algorithm work out more neatly.)

Now we do the following.

1. We find the vertex v in the network that has the smallest estimated distance from s, i.e., the smallest distance about which we are not yet certain.
2. We mark this distance as being certain.
3. We calculate the distances from s via v to each of the neighbors of v by adding to v's distance the lengths of the edges connecting v to each neighbor. If any of the resulting distances is smaller than the current estimated distance to the same neighbor, the new distance replaces the older one.
4. We repeat from step 1 until the distances to all vertices are flagged as being certain.

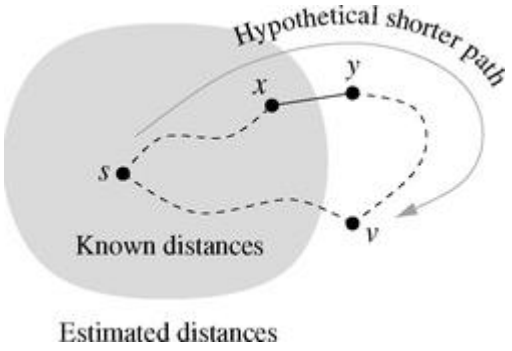


Figure 10.5: Paths in Dijkstra's algorithm. If v is the vertex with the smallest estimated (i.e., not certain) distance from s then that estimated distance must in fact be the true shortest distance to v. If it were not and there were a shorter path $s, \ldots, x, y, \ldots, v$ then all points along that path must have shorter distances from s than v's estimated distance, which means that y has a smaller estimated distance than v, which is impossible.

Simple though it is to describe, it's not immediately obvious that this algorithm does what it is supposed to do and finds true shortest paths. The crucial step is step 2 where we declare the current smallest estimated distance in fact to be certain. That is, we claim that among vertices for which we don't yet definitely know the distance, the smallest distance recorded to any vertex is in fact the smallest possible distance to that vertex.

To see why this is true consider such a vertex, which we'll again call v, and consider a hypothetical path from s to v that has a shorter length than the current estimated distance recorded for v. The situation is illustrated in Fig. 10.5. Since this hypothetical path is shorter than the estimated distance to v, the distance along the path to each vertex in the path must also be less than that estimated distance.

Furthermore, there must exist somewhere along the path a pair of adjacent vertices x, y such that

x's distance is known for certain and y's is not. Vertex x need not necessarily be distinct from vertex s (although we have drawn it that way in the figure), but vertex y must be distinct from v: if y and v were the same vertex, so that v was a neighbor of x, then we would already have found the shorter path to v when we explored the neighbors of x in step 3 above and we would accordingly have revised our estimate of v's distance downward. Since this hasn't happened, y and v must be distinct vertices.

But notice now that y's current estimated distance will be at most equal to its distance from s *along the path* because that distance is calculated in step 3 above when we explore x's neighbors. And since, as we have said, all distances along the path are necessarily less than the current estimated distance to v, it follows that y's estimated distance must be less than v's and we have a contradiction, because v is by hypothesis the vertex with the shortest estimated distance. Hence there is no path to vertex v with length less than v's current estimated distance, so we can safely mark that distance as being certain, as in step 2 above.

Thus on each step the algorithm correctly flags one additional distance as being known exactly and when all distances have been so flagged the algorithm has done its job.

As with breadth-first search, the running time of Dijkstra's algorithm depends on how it is implemented. The simplest implementation is one that searches through all vertices on each round of the algorithm to find the one that has the smallest estimated distance. This search takes time $O(n)$. Then we must calculate a new estimated distance to each of the neighbors of the vertex we find, of which there are $O(m/n)$ on average. To leading order, one round thus takes time $O(m/n + n)$ and the whole algorithm, which runs (in the worst case of a network with a single component) for n rounds, takes time $O(m + n^2)$ to find the distance from s to every other vertex.

But we can do better than this. If we store the estimated distances in a binary heap (see Section 9.7) then we can find the smallest one and remove it from the heap in time $O(\log n)$. The operation of replacing an estimated distance with a new and better estimate (which in the worst case we have to do an average of $O(m/n)$ times per round) also takes $O(\log n)$ time, and hence a complete round of the algorithm takes time $O((m/n)\log n + \log n)$ and all n rounds then take $O((m+n)\log n)$, or $O(n \log n)$ on a sparse network with $m \propto n$. This is very nearly the best running time known for this problem,[156] and close to, though not quite as good as, the $O(m + n)$ for the equivalent problem on an unweighted network (factors of $\log n$ being close to constant given that the logarithm is a very slowly growing function of its argument).

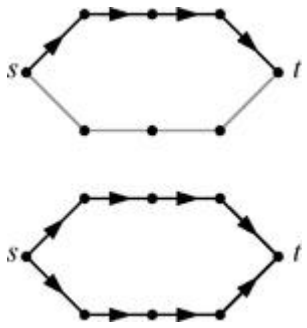
As we have described it, Dijkstra's algorithm finds the shortest distance from a vertex s to every other in the same component but, like breadth-first search, it can be modified also to find the actual paths that realize those distances. The modification is very similar to the one for breadth-first search. We maintain a shortest path tree, which is initially empty and to which we add directed edges pointing from the vertices along the first step of their shortest path to s. We create such a directed edge when we first assign a vertex an estimated distance less than ∞ and move the edge to point to a new vertex every time we find a new estimated distance that is less than the current one. The last position in which an edge comes to rest indicates the true first step in the shortest path. If a new estimate of the distance to a vertex is ever exactly the same as the current estimate then we put two directed edges in the shortest path tree indicating the two alternative paths that give the shortest distance. When the algorithm is finished the shortest path tree, like those in Fig. 10.2, can be used to reconstruct the shortest paths themselves, or to calculate other quantities such as a weighted version of betweenness centrality (which could be used for instance as a measure of traffic flow in a network where traffic always takes the shortest weighted path).

10.5 MAXIMUM FLOWS AND MINIMUM CUTS

In Section 6.12 we discussed the ideas of connectivity, independent paths, cut sets, and maximum flows in networks. In particular, we defined two paths that connect the same vertices *s* and *t* to be edge-independent if they share none of the same edges and vertex-independent if they share none of the same vertices except for *s* and *t* themselves. And the edge or vertex connectivity of the vertices is the number of edge–or vertex-independent paths between them. We also showed that the edge or vertex connectivity is equal to the size of the minimum edge or vertex cut set—the minimum number of edges or vertices that need to be removed from the network to disconnect *s* from *t*. Connectivity is thus a simple measure of the robustness of the connection between a pair of vertices. Finally, we showed that the edge-connectivity is also equal to the maximum flow that can pass from *s* to *t* if we think of the network as a network of pipes, each of which can carry one unit of flow.

In this section we look at algorithms for calculating maximum flows between vertices on networks. As we will see, there is a simple algorithm, the Ford-Fulkerson or augmenting path algorithm, that calculates the maximum flow between two vertices in average time $O((m + n)m/n)$. Once we have this maximum flow, then we also immediately know the number of edge-independent paths and the size of the minimum edge cut set between the same vertices. With a small extension, the algorithm can also find the particular edges that constitute the minimum edge cut set. A simple modification of the augmenting path algorithm allows us also to calculate vertex-independent paths and vertex cuts sets.

All the developments of this section are described for undirected networks, but in fact the algorithms work perfectly well, without modification, for directed networks as well. Readers who want to know more about maximum flow algorithms are recommended to look at the book by Ahuja *et al.* [8], which contains several hundred pages on the topic and covers almost every conceivable detail.



A simple breadth-first search finds a path from source *s* to target *t* (top) in this network. A second search using only the edges not used in the first finds a second path (bottom).

10.5.1 THE AUGMENTING PATH ALGORITHM

In this section we describe the augmenting path algorithm of Ford and Fulkerson for calculating maximum flows between vertices in a network.[157] The case of primary interest to us is the one where each edge in the network can carry the same single unit of flow. The algorithm can be used in the more general case where the edges have varying capacities, but we will not discuss that case here.[158]

The basic idea behind the augmenting path algorithm is a simple one. We first find a path from source *s* to target *t* using the breadth-first search algorithm of Section 10.3.[159] This "uses up" some of the edges in the network, filling them to capacity so that they can carry no more flow. Then we find another path from *s* to *t* among the remaining edges and we repeat this procedure until no more paths can be found.

Unfortunately, this does not yet give us a working algorithm, because as we have described it the procedure will not always find the maximum flow. Consider Fig. 10.6a. If we apply breadth-first search between *s* and *t* we find the path marked in bold. Unfortunately, once we have filled all the edges along this path to capacity there are no more paths from *s* to *t* that can be constructed with the remaining edges, so the algorithm stops after finding just one path. It is clear, however, that there are in fact two edge-independent paths from *s* to *t*—along the top and bottom of the network—and a maximum flow of two, so the algorithm has given the wrong answer.

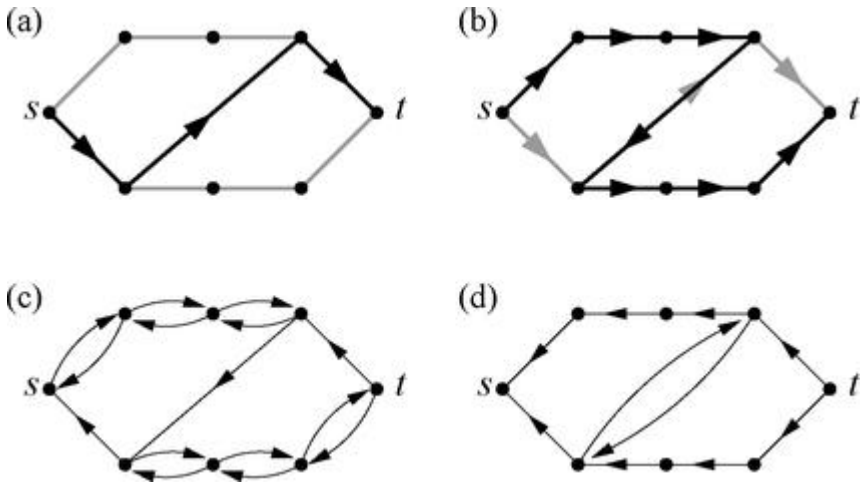


Figure 10.6: The augmenting path algorithm. (a) We find a first path from source *s* to target *t* using breadth-first search. This leaves no more independent paths from *s* to *t* among the remaining edges. (b) However, if we allow flows in both directions along an edge (such as the central edge in this network), then we can find another path. Panels (c) and (d) show the residual graphs corresponding to panels (a) and (b).

There is however, a simple fix for this problem, which is to allow fluid to flow simultaneously *both ways* down an edge in our network. That is, we allow a state in which there is one unit of flow in each direction along any given edge. If the edges were real pipes, then this would not be possible: if a pipe is full of fluid flowing one way, then there is no room for fluid flowing the other way too. However, if fluid *were* flowing both ways down an edge, the net flow in and out of either end of that edge would be zero—the two flows would effectively cancel out giving zero net flow.

And zero flow down an edge certainly is possible.

So we use a trick and allow our algorithm to place a unit of flow both ways down any edge, but declare this to mean in practice that there is no flow at all on that edge. This means that the paths we will find will no longer necessarily be independent paths, since two of them can share an edge so long as they pass along it in opposite directions. But this doesn't matter: the flows the paths represent are still allowed flows, since no pipe is ever required to carry more than one unit of flow, and we know that in the end our final, maximum flow will be numerically equal to the actual number of independent paths, even though those independent paths may be different from the paths picked out by the algorithm. Thus we create an algorithm that counts independent paths by counting a special class of *non*-independent paths: strange as this sounds, the max-flow/min-cut theorem tells us that it must work, and indeed it does.

More generally, since the maximum allowed flow down an edge is one unit in either direction, we can have any number of units flowing either way down an edge provided they cancel out so that net flow is no more than one unit. Thus, two units of flow in either direction would be allowed, or three units one way and four the other, and so forth. Three units one way and five the other would not be allowed, however.[160]

To see how this works in practice, consider Fig. 10.6 again. We begin by performing a breadth-first search that finds the path shown in panel (a). Now, however, there is a second path to be found, as shown in panel (b), making use of the fact that we are still allowed to send one unit of flow *backwards* along the edge in the center of the network. After this, however, there are no more paths left from *s* to *t* and so the algorithm stops and tells us that the maximum possible flow is two units, which is the correct answer.

This is merely one example of the algorithm: we still have to prove that it gives the correct answer in all cases, which we do in Section 10.5.3. To understand the proof, however, we first need to understand how the algorithm is implemented.

10.5.2 IMPLEMENTATION AND RUNNING TIME

Implementation of the augmenting path algorithm makes use of a *residual graph*, which is a directed network in which the edges connect the pairs of vertices on the original network between which we still have capacity available to carry one or more units of flow in the given direction. For instance, Figs. 10.6c and 10.6d show the residual graphs corresponding to the flow states in 10.6a and 10.6b.

The residual graph is constructed by first taking the initial network and replacing each undirected edge with two directed ones, one in each direction. We now perform our breadth-first searches on this residual graph, rather than on the original network, respecting the directions of the edges. Every time our algorithm finds a new path through the network, we update the residual graph by adding a directed edge in the opposite direction to the path between every pair of vertices along the path, provided no such edge already exists. If a vertex pair already has such a backward-pointing edge, we instead take away a forward-pointing one. (There will always be such a forward-pointing edge, otherwise the path would not exist in the first place.) The largest number of edges we update during this process is m, the total number of edges in the original network, so the process takes time $O(m)$ and thus makes no difference to the $O(m + n)$ time complexity of the breadth-first search.

Now we find the next path by performing another breadth-first search on the updated residual graph. By always working on the residual graph in this way, we insure that we find only paths along edges that have not yet reached their maximum flow. Such paths are called *augmenting paths*. The process is repeated until our breadth-first search fails to find any augmenting path from s to t, at which point we have found all the paths there are and the number of paths found is equal to the number of units in the maximum flow from s to t.

Each breadth-first search, along with the corresponding updates to the residual graph, takes time $O(m + n)$ for a network stored in adjacency list format (see Sections 9.4 and 10.3). Moreover, the number of independent paths from s to t can be no greater than the smaller of the two degrees k_s and k_t of the source and target vertices (since each path must leave or enter one of those vertices along some edge, and that edge can carry at most one path). Thus the running time of the algorithm is $O\ \min(k_s, k_t)(m + n))$. If we are interested in the average running time over many pairs of vertices, then we can make use of the fact that $\langle \min(k_s, k_t) \rangle \leq \langle k \rangle$ (where the averages are over all vertices), and recalling that $\langle k \rangle = 2m/n$ (Eq. (6.23)), this implies that the average running time of the algorithm is $O((m + n)m/n)$, which is $O(n)$ on a sparse network with $m \propto n$. (On the other hand, on a dense graph where $m \propto n^2$, we would have $O(n^3)$, which is much worse.)

10.5.3 WHY THE ALGORITHM GIVES CORRECT ANSWERS

It is plausible but not immediately obvious that the augmenting path algorithm correctly finds maximum flows. We can prove that it does as follows.

Suppose at some point during the operation of the algorithm (including the very beginning) we have found some (or no) paths for flow from s to t, but any paths we have found do not yet constitute the maximum possible flow. That is, there is still room in the network for more flow from s to t. If this is the case then, as we will now show, there must exist at least one augmenting path from s to t, which by definition carries one unit of flow. And if there exists an augmenting path, our breadth-first search will always find it, and so the algorithm will go on finding augmenting paths until there is no room in the network for more flow, i.e., we have reached the maximum flow, which is equal to the number of paths found.

Thus the proof that the algorithm is correct requires only that we prove the following theorem:

> If at some point in our algorithm the flow from s to t is less than the maximum possible flow, then there must exist at least one augmenting path on the current residual graph.

Consider such a point in the operation of the algorithm and consider the flows on the network as represented by f, the set of all individual net flows along the edges of the network. And consider also the maximum possible flow from s to t, represented by $f_{\max}$, the corresponding set of individual net flows. By hypothesis, the total flow out of s and into t is greater in $f_{\max}$ than in f. Let us calculate the difference flow $\Delta f = f_{\max} - f$, by which we mean we subtract the net flow along each edge in f from the net flow along the corresponding edge in $f_{\max}$, respecting flow direction—see Fig. 10.7. (For instance, the difference of two unit flows in the same direction would be zero while the difference of two in opposite directions would be two in one direction or the other.)

Since the total flow is greater in $f_{\max}$ than in f, the difference flow Δf must have a net flow out of s and net flow into t. What's more, because the "fluid" composing the flow is conserved at vertices, every vertex except s and t must have zero net flow in or out of it in both $f_{\max}$ and f and hence also in Δf. But if each vertex other than s and t has zero net flow, then the flow from s to t must form at least one path across the network—it must leave every vertex it enters, except the last one, vertex t. Let us choose any one of these paths formed by the flow from s to t in Δf and let us call this path p.

Since there is a positive flow in Δf in the forward direction along each edge in p, there must have been no such flow in f along any of the same edges. If there were such a flow in f then when we performed the subtraction $\Delta f = f_{\max} - f$ the flow in Δf would be either zero or negative on the edge in question (depending on the flow in $f_{\max}$), but could not be positive. Thus we can always safely add to f a unit of flow forward along each edge in p without overloading any of the edges. But this immediately implies that p is an augmenting path for f.

Thus, for any flow that is not maximal, at least one augmenting path always exists, and hence it follows that the augmenting path algorithm as described above is correct and will always find the maximum flow.

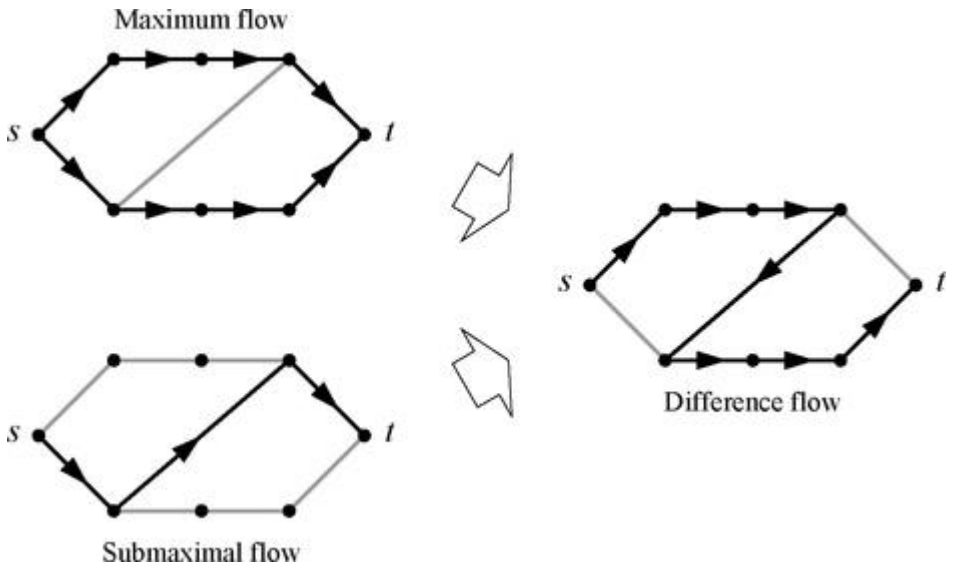


Figure 10.7: Correctness of the augmenting path algorithm. If we subtract from the maximum flow $f_{\max}$ (upper left) any submaximal flow f (lower left), the resulting difference flow (right) necessarily contains at least one path from s to t, and that path is necessarily an augmenting path for f.

10.5.4 FINDING INDEPENDENT PATHS AND MINIMUM CUT SETS

Once we have found the maximum possible flow between a given pair of vertices, we also automatically have the size of the minimum edge cut set and the number of edge-independent paths, which are both numerically equal to the number of units in the maximum flow (see Section 6.12).

We might also wish to know exactly where the independent paths run. The augmenting path algorithm does not give us this directly since, as we have seen, the augmenting paths it finds are not necessarily the same as the independent paths, but only a very small extension of the algorithm is necessary to find the independent paths: we take the final residual graph produced at the end of the algorithm and remove from it every pair of directed edges that joins the same two vertices in opposite directions—see Fig. 10.8. In other words we are removing all network edges that carry no net flow. The edges remaining after we have done this are necessarily those that actually carry the maximum flow and it is a straightforward matter to trace these edges from *s* to *t* to reconstruct the paths themselves.[161] (In fact, as Fig. 10.8 shows, the remaining directed edges in the residual graph point backwards from *t* to *s*, so it is often easier to reconstruct the paths backwards.)

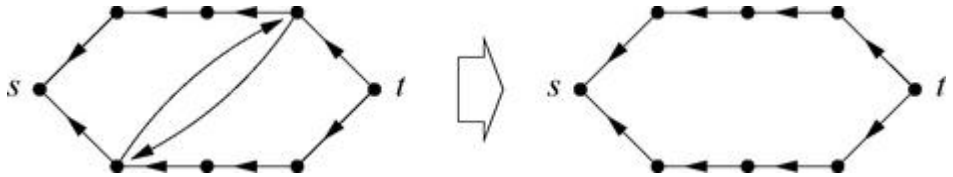


Figure 10.8: Reconstructing the independent paths from the residual graph. Deleting every pair of edges on the residual graph that join the same two vertices in opposite directions leaves a graph consisting of the independent paths only, spelled out in directed edges that point backwards along those paths from target to source.

Another thing we might want is the set of edges that constitutes the minimum cut set for the vertices *s* and *t*. In fact in most cases there is more than one cut set of the minimum size, so more generally we would like to find one of the minimum cut sets. Again we can do this by a small extension of the augmenting path algorithm. The procedure is illustrated in Fig. 10.9. We again consider the final residual graph generated at the end of the algorithm. By definition this graph has no directed path in it from *s* to *t* (since if it did the algorithm would not have stopped yet). Thus we can reach some subset of vertices by starting at vertex *s*, but we cannot reach all of them. (For example, we cannot reach *t*.) Let V_s be the subset of vertices reachable from *s* by some path on the residual graph and let V_t be the set of all the other vertices in the graph that are not in V_s. Then the set of edges on the original graph that connect vertices in V_s to vertices in V_t constitutes a minimum cut set for *s* and *t*.

Why does this work? Clearly if we removed all edges that connect vertices in V_s to those in V_t we disconnect *s* and *t*, since then there is no path at all between *s* and *t*. Thus the edges between V_s and V_t constitute a cut set. That it is a *minimum* cut set we can see by the following argument. Every edge from a vertex in V_s to a vertex in V_t must be carrying a unit of flow from V_s to V_t. If it were not, then it would have available capacity away from V_s, meaning that there would be a

corresponding directed edge away from V_s in the residual graph. In that case, however, the vertex at the far end of that edge would be reachable from V_s on the residual graph and therefore would be a part of V_s. Since the vertex in question is, by hypothesis, in V_t and not in V_s, it follows that it must be carrying a unit of the maximum flow from *s* to *t*.

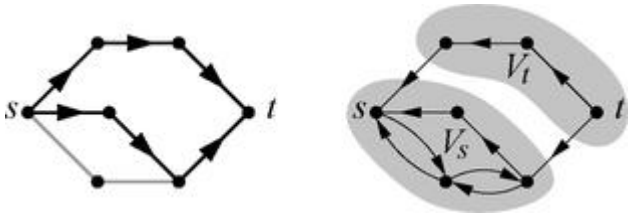


Figure 10.9: Finding a minimum cut set. Once we have found a set of maximum flows for a given *s* and *t* (left) we can find a corresponding minimum cut set by considering the residual graph (right). The set V_s is the set of vertices reachable from *s* by following directed edges on the residual graph and V_t is the rest of the vertices. The minimum cut set is the set of edges (two of them in this case) that connect V_s to V_t on the original network.

Now, since every edge in the cut set between V_s and V_t is carrying a unit of flow, the size of that cut set is numerically equal to the size of the flow from V_s to V_t, which is also the flow from *s* to *t*. And, by the max-flow/min-cut theorem, a cut set between *s* and *t* that is equal in size to the maximum flow between *s* and *t* is a minimum cut set, and hence our result is proved.

10.5.5 FINDING VERTEX-INDEPENDENT PATHS

Once we know how to find edge-independent paths it is straightforward to find vertex-independent paths as well. First, note that any set of vertex-independent paths between two vertices *s* and *t* is necessarily also a set of edge-independent paths: if two paths share none of the same vertices, then they also share none of the same edges. Thus, we can find vertex-independent paths using the same algorithm that we used to find edge-independent paths, but adding the restriction that no two paths may pass through the same vertex. One way to impose this restriction is the following. First, we replace our undirected network with a directed one, as shown in Fig. 10.10, with a directed edge in either direction between every connected pair of vertices. This does not change the maximum flow possible in the network and hence does not change the number of independent paths either.

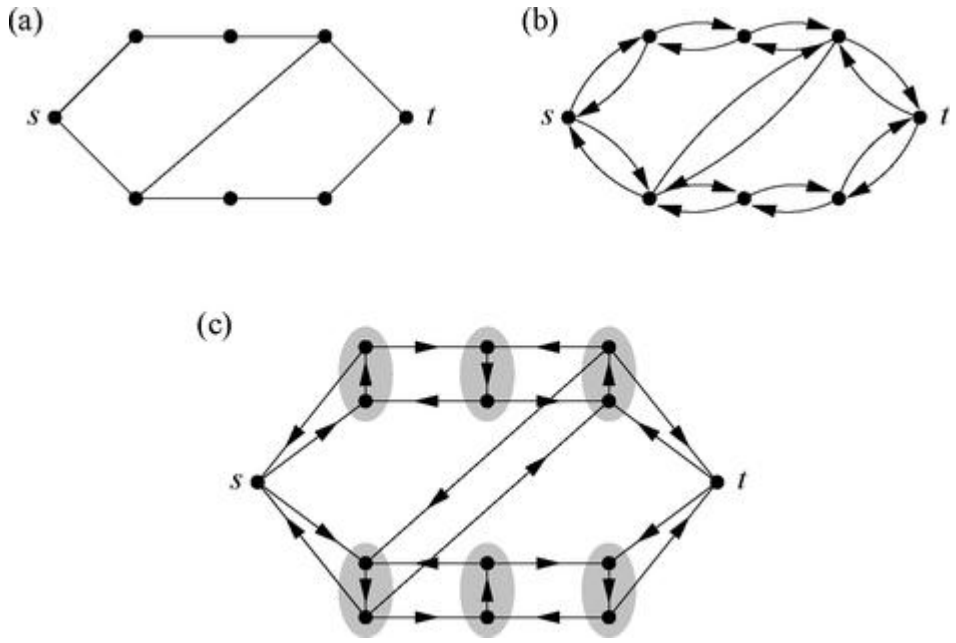


Figure 10.10: Mapping from the vertex-independent path problem to the edge-independent path problem. Starting with an undirected network (a), we (b) replace each edge by two directed edges, then (c) replace each vertex, except for *s* and *t*, with a pair of vertices with a directed edge between them (shaded) following the prescription in Fig. 10.11. Edge-independent paths on the final network then correspond to vertex-independent paths on the initial network.

Second, we replace each of the vertices in the network, except *s* and *t*, with a construct like that shown in Fig. 10.11. Each vertex is replaced with two vertices separated by a directed edge. All original incoming edges connect to the first of these two (on the left in Fig. 10.11) and all outgoing edges to the second. This new construct functions as the original vertex did, allowing flows to pass in along ingoing edges and out along outgoing ones, but with one important difference: assuming that the new edge joining the two vertices has unit capacity like all others, we are now limited to just one unit of flow through the entire construct, since every path through the construct must traverse this central edge. Thus every allowed flow on this network corresponds to a flow on the original network with at most a single unit passing though each vertex.

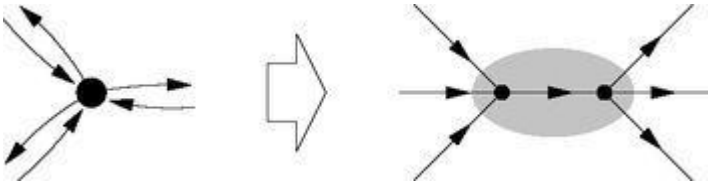


Figure 10.11: Vertex transformation for the vertex-independent path algorithm. Each vertex in the network is replaced by a pair of vertices joined by a single directed edge. All incoming edges are connected to one of the pair and all outgoing edges to the other as shown.

Transforming the entire network of Fig. 10.10a using this method gives us a network that looks like Fig. 10.10c. Now we simply apply the normal augmenting path algorithm to this directed network, and the number of *edge*-independent paths we find is equal to the number of *vertex*-independent paths on the original network.

PROBLEMS

10.1 What is the time complexity, as a function of the number n of vertices and m of edges, of the following network operations if the network in question is stored in adjacency list format?

a. Calculating the mean degree.
b. Calculating the median degree.
c. Calculating the air-travel route between two airports that has the shortest total flying time, assuming the flying time of each individual flight is known.
d. Calculating the minimum number of routers that would have to fail to disconnect two given routers on the Internet.

10.2 For an undirected network of n vertices stored in adjacency list format show that:

a. It takes time $O(n(n + m))$ to find the diameter of the network.
b. It takes time $O(\Box k\Box)$ on average to list the neighbors of a vertex, where $\Box k\Box$ is the average degree in the network, but time $O(\Box k^2\Box)$ to list the second neighbors.

10.3 For a directed network in which in–and out-degrees are uncorrelated, show that it takes time $O(m^2/n)$ to calculate the reciprocity of the network. Why is the restriction to uncorrelated degrees necessary? What could happen if they were correlated?

10.4 Suppose that we define a new centrality measure x_i for vertex i in a network to be a sum of contributions as follows: 1 for vertex i itself, α for each vertex at (geodesic) distance 1 from i, α^2 for each vertex at distance 2, and so forth, where $\alpha < 1$ is a given constant.

a. Write an expression for x_i in terms of α and the geodesic distances d_{ij} between vertex pairs.
b. Describe briefly an algorithm for calculating this centrality measure. What is the time complexity of calculating x_i for all i?